

Team Name

LEGAL VISIONARIES

Project Title

LegalEase – AI-Powered Legal Document Generator

Team Members

1. **Rithika**
2. **Sindhuja**
3. **Sumathi**
4. **Sathvika**

PHASE 5: PROJECT DEVELOPMENT

1. Introduction

The Project Development phase focuses on implementing the LegalEase – AI-Powered Legal Document Generator according to the requirements and system design prepared during the previous phases.

The project is developed using Python-based technologies and follows a modular structure that separates AI integration, frontend development, backend API development, and document formatting.

The current project workspace is maintained separately from the SmartBridge documentation and contains the source code and development environment.

2. Development Workspace

The actual LegalEase application is maintained in the following project directory:

Project Location:

D:\LegalEaseAI

The project workspace contains the application source code, configuration files, virtual environment, and supporting project files.

3. Current Project Structure

The implemented project currently contains the following major components:

LegalEaseAI/

|

| └─ ai_core/

|

| └─ __init__.py

|

| └─ gemini_generator.py

|

| └─ frontend/

|

| └─ static/

|

| └─ style.css

|

| └─ __init__.py

|

| └─ app.py

|

```
├── LegalEaseAPI/
│   ├── __init__.py
│   ├── main.py
│   └── routes.py
│
├── utils/
│   ├── __init__.py
│   └── formatter.py
│
├── venv/
│
├── .env
├── .env.example
├── .gitignore
├── README.md
├── requirements.txt
└── run.py
```

The structure separates the major responsibilities of the application and provides a foundation for further development and integration.

4. AI Integration Module

The `ai_core` folder contains the AI-related components.

File:

`ai_core/gemini_generator.py`

This module is responsible for the integration of the Gemini AI service and the generation of legal document content.

The AI integration is separated from the frontend and backend so that the AI functionality can be maintained independently.

The module uses configuration values supplied through environment variables.

5. Frontend Module

The frontend folder contains the user-interface components.

Main File:

frontend/app.py

Supporting File:

frontend/static/style.css

The frontend is designed to provide the user with an interface for interacting with the LegalEase application.

The frontend is responsible for collecting document-related information and displaying the generated result.

The CSS file provides styling for the application interface.

Responsible Team Member: Rithika – Frontend / UI

6. Backend API Module

The LegalEaseAPI folder contains the backend components.

Main Files:

LegalEaseAPI/main.py

LegalEaseAPI/routes.py

The backend is developed using FastAPI.

Its responsibilities include:

- Starting the backend application.
- Defining API routes.
- Receiving document-generation requests.
- Processing requests.
- Connecting the application with the required generation components.

The document-generation workflow is designed around the /generate API endpoint.

Responsible Team Member: Sindhuja – Backend

7. Document Formatting Module

The utils folder contains supporting utility functionality.

Main File:

utils/formatter.py

This component is responsible for document-formatting functionality required by the application.

The formatter supports the processing of generated content for document output and presentation.

8. Frontend and Backend Workflow

The current application follows the planned architecture:

User

↓

Streamlit Frontend

↓

FastAPI Backend

↓

Gemini AI Integration

↓

Generated Legal Document

↓

Formatting

↓

Preview / Export

This structure separates user interaction from backend processing and AI generation.

9. Environment Configuration

The project contains an environment configuration file:

.env

The .env file is used for storing sensitive configuration values such as the Gemini API key and other environment-specific settings.

An .env.example file is also maintained as a template for required environment variables.

Sensitive API keys must not be included in the public GitHub repository.

10. Virtual Environment

The project contains a Python virtual environment:

venv

The virtual environment provides an isolated environment for installing and managing the Python packages required by the LegalEase application.

This helps prevent conflicts between project dependencies and other Python projects on the computer.

11. Dependency Management

The project contains:

requirements.txt

This file is used to maintain the Python packages required by the application.

The dependencies support the major components of the project, including:

- FastAPI
- Uvicorn
- Streamlit
- Gemini AI integration
- Document generation
- PDF generation
- Environment configuration

12. Application Entry Point

The project contains:

run.py

This file is used as a project-level entry point for running the application workflow.

It provides a centralized way to start the required application components.

13. Version Control and Project Files

The project contains:

.gitignore

The .gitignore file is used to prevent unnecessary or sensitive files from being committed to GitHub.

Examples include:

- Virtual environment files
- Python cache files
- Environment files containing secrets

The project also contains:

README.md

The README provides information about the project and its setup and usage.

14. Development Responsibilities

Rithika – Frontend / UI

Responsible for:

- Streamlit interface
- User input components
- UI styling
- Document preview interface
- Frontend improvements

Sindhuja – Backend

Responsible for:

- FastAPI application
- API routes
- Request processing
- Backend integration
- API functionality

Sumathi – AI Integration

Responsible for:

- Gemini AI integration
- AI generation module
- Prompt preparation
- AI response handling

- AI and backend integration

Sathvika – Testing & Documentation

Responsible for:

- Functional testing
- Integration testing
- Recording test results
- Maintaining project documentation
- Capturing development screenshots
- Preparing SmartBridge phase documentation

15. Development Evidence

Development evidence will be maintained separately from the source-code workspace.

The SmartBridge Phase 5 evidence folder is:

D:\LegalEase\Phase-5-Project-Development\Screenshots

Screenshots will document important development stages such as:

1. Project structure
2. Frontend implementation
3. Backend implementation
4. AI integration
5. Application interface
6. Document generation
7. Document export
8. Successful application execution

Screenshots will be added progressively as development and testing are completed.

16. Source Code Management

The actual source code is maintained separately in:

D:\LegalEaseAI

The source code includes:

- AI integration

- Frontend
- Backend API
- Formatting utilities
- Application entry point
- Dependency configuration

The SmartBridge documentation folder contains the project reports and evidence rather than duplicate copies of the source code.

17. Security Considerations

Sensitive information such as API keys must not be shared through screenshots, public repositories, or project documentation.

The .env file should remain local and should be excluded from GitHub through .gitignore.

Only the .env.example template should be shared when necessary, without real secret values.

18. Development Outcome

The current project structure establishes the foundation for the LegalEase application.

The project contains separate components for:

- AI integration
- Frontend/UI
- Backend/API
- Document formatting
- Environment configuration
- Dependency management

The next development activities will focus on integrating and validating these components, testing the complete document-generation workflow, and ensuring that the required document preview and export functionality works correctly.

19. Conclusion

The Project Development phase establishes the actual implementation structure of LegalEase.

Unlike the previous planning phases, this phase contains the application's real source-code workspace and development environment.

The modular structure allows the four team members to work according to their assigned responsibilities while keeping the application organized and maintainable.

The completed development components will be verified during the **Phase 6 – Project Testing** stage.